Event Loop and SGLang

HJ

November 7, 2026

1 Event Loop Models

A service is typically built around an event loop: a main loop receives requests and dispatches them to workers, which do the real work and asynchronously return a completion signal.

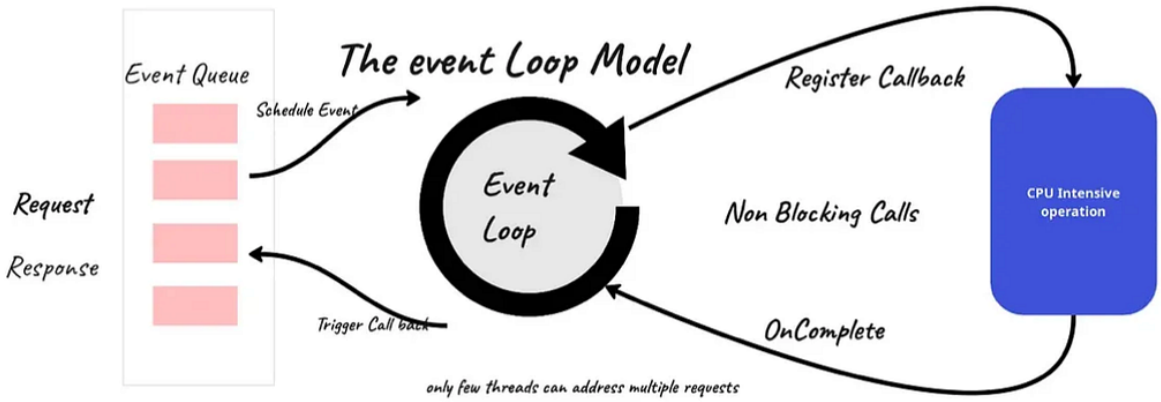


Figure 1: Event loop: the main loop launches workers; workers return asynchronously.

Why asynchronous? Two classic cases:

- **Network**: many connections, each I/O may block for a long and unpredictable time; while one is blocked, the CPU can serve others.
- **UI**: blocking the main thread drops frames; a single block is a visible freeze.

2 Event Loop in SGLang

SGLang's scheduler is a single-threaded event loop with two modes:

- **overlap (async, default)**: the CPU prepares and samples batch $N{+}1$ while the GPU runs batch N, via separate CUDA streams.
- **normal (sync)**: the CPU waits for each forward to finish; a fallback used where overlap is unsupported.

The CPU still has real per-step work — scheduling, sampling, and pre/post-processing — not "nothing to do". The overlap scheduler exists to hide exactly this CPU work behind the GPU forward. Speculative decoding is a typical case that amplifies it: a draft-model forward, verification, and accept/reject bookkeeping are added on top of the normal step.

2.1 Various Parallelism

To serve a model larger than one GPU, the model or data is sharded along some axis:

- **DP (Data Parallelism)**: shard the batch; each replica serves its own requests independently.
- **TP (Tensor Parallelism)**: shard the hidden dimension; one forward spans all GPUs with all-reduce.
- **EP (Expert Parallelism)**: shard the experts (MoE); tokens are dispatched via all-to-all.
- **PP (Pipeline Parallelism)**: shard the layers; a batch flows through the stages.